Apuntes de PHP

Desarrollo de Aplicaciones Web en Entorno Servidor

Unidad 2 - Programación con PHP 8
Guía de consulta para hacer ejercicios y para el examen

Contenido: resumen de la teoría de la unidad, funciones más usadas, patrones de formularios, ficheros, excepciones y POO, ejercicios 1-19 resueltos y el examen de recuperación (12/11/2024) resuelto de principio a fin. Incluye una lista de errores típicos y una lista de comprobación antes de entregar.

Parte	Qué encontrarás
1. Fundamentos	Sintaxis, variables, tipos, operadores, estructuras de control
2. Arrays y funciones	Arrays escalares/asociativos, funciones propias, funciones de PHP más usadas
3. Formularios	GET/POST, recordar datos, validar, subir ficheros, formularios en pasos
4. Ficheros y excepciones	Leer/escribir ficheros de texto con ;, try/catch, excepciones propias
5. POO	Clases, constructor, herencia, abstractas, interfaces, namespaces
6. Expresiones regulares	preg_match y patrones típicos de validación
7. Ejercicios resueltos	Ejercicios 3-19 del tema y el juego de capitales
8. Examen modelo	Venta de entradas (Cultura Navalmoral): ejercicios 1, 2 y 3 completos
9. Errores y checklist	Por qué no funciona, erratas de las diapositivas y lista antes de entregar

1. Fundamentos del lenguaje

1.1 Qué es PHP y cómo se ejecuta

- PHP es un lenguaje **interpretado en el servidor**: el navegador nunca ve el código PHP, solo el HTML que genera.
- Los ficheros tienen extensión `.php` y deben ejecutarse a través del servidor (XAMPP): `http://localhost/carpeta/fichero.php`. Si abres el fichero con doble clic (`file://`) no se ejecuta.
- En XAMPP los ficheros van dentro de `C:\xampp\htdocs\`. El fichero de configuración es `C:\xampp\php\php.ini`.
- Es **case sensitive** en variables y constantes (`$Edad` y `$edad` son distintas). Las funciones y palabras reservadas (`echo`, `IF`) no distinguen mayúsculas, pero escríbelas siempre en minúsculas.
- Cada instrucción termina en `;`.

1.2 Delimitadores y comentarios

```
<?php
  // instrucciones PHP
?>
<?= $nombre ?>          <!-- atajo de <?php echo $nombre; ?> (muy útil dentro de HTML) -->

// comentario de una línea
#  otro comentario de una línea
/* comentario
   de varias líneas */
```

> **OJO:** Si el fichero es solo PHP puedes omitir el `?>` final: evita espacios en blanco que dan el error 'headers already sent'. La etiqueta corta `<? ?>` depende del php.ini; usa siempre `<?php`.

1.3 Variables

- No se declaran ni se indica el tipo: lo decide el valor. El nombre empieza por `$` seguido de letra o `_`.
- Distinguen mayúsculas: `$Edad` y `$edad` son variables distintas.

```
$edad = 23;              // integer
$nombre = "María";       // string (comillas dobles: interpreta variables)
$apellido = 'Martín';    // string (comillas simples: texto literal)
$estatura = 1.60;        // float -> gettype() devuelve "double"
$alumno = true;          // boolean
echo gettype($edad), " ", gettype($estatura);    // integer double
```

1.4 Mostrar información (generar HTML)

Instrucción	Para qué sirve
`echo $a, $b;`	Admite varios parámetros separados por coma. No devuelve valor.
`print($texto);`	Solo un parámetro.
`printf(formato, ...)`	Escribe texto con formato. `%s` cadena, `%d` entero, `%.2f` decimal con 2 decimales, `%05d` entero con ceros.
`sprintf(formato, ...)`	Igual que printf pero **devuelve** la cadena (no la imprime). Útil para no concatenar.
`var_dump($x)` / `print_r($x)`	Muestran tipo y contenido; imprescindibles para depurar arrays. Con `<pre>` se ven ordenados.

```
echo "<h2>Me llamo $nombre $apellido</h2>";        // interpolación en comillas dobles
echo 'Tengo ' . $edad . ' años<br/>';              // concatenación con el punto
echo "Edad: {$datos['edad']}";                     // elementos de array: usar llaves
printf("%s mide %.2f m<br/>", $nombre, $estatura); // María mide 1.60 m
$txt = sprintf("%05.1f", 3.14159);                 // "003.1"
echo '<pre>'; print_r($array); echo '</pre>';
```

Cadenas: comillas simples y dobles

- `"$variable"` sustituye la variable por su valor; `'$variable'` la escribe literalmente.
- Dentro de comillas simples solo funcionan los escapes `\\` y `\'`. Dentro de dobles: `\n`, `\t`, `\"`, `\$`, `\\`.
- El salto de línea `\n` **no se ve en el navegador**: para verlo usa `<br/>`. `PHP_EOL` es el salto de línea del sistema (útil en ficheros).

1.5 Tipos de datos

Tipo	Descripción
integer	Número sin decimales. Decimal (`10`), octal (`012`), hexadecimal (`0x1A`), binario (`0b101`).
float (double)	Número con decimales o notación científica (`1.5e3`).
string	Cadena entre comillas simples o dobles.
boolean	`true` / `false`. Se consideran false: `0`, `0.0`, `""`, `"0"`, `[]`, `null`.
null	Sin valor.
array	Conjunto de pares clave => valor. Primera posición: 0.
object	Instancia de una clase.

1.6 Conversiones de tipo

PHP convierte automáticamente cuando lo necesita (`1 + "10"` da 11). También puedes forzar la conversión (casting) poniendo el tipo entre paréntesis delante de la variable.

```
$mi_entero = 3;  $mi_real = 2.3;
$r1 = $mi_entero + $mi_real;          // 5.3  (float)
$r2 = $mi_entero + (int)$mi_real;     // 5    (int; el decimal se descarta, no se redondea)
$n = (int)"25abc";                    // 25
$s = (string)12;   $b = (bool)"0";    // "12"  false
$f = (float)"3.5";  $a = (array)"x";  // 3.5   ["x"]
settype($var, "integer");             // cambia el tipo de la propia variable
intval("12"), floatval("1.5"), strval(5), boolval(0)
```

1.7 Ámbito de las variables

- **Locales**: una variable definida en una función solo existe dentro de ella; una definida fuera no es visible dentro de las funciones.
- **global**: `global $x;` dentro de una función para usar la variable global (también `$GLOBALS['x']`). Es mejor pasar parámetros.
- **static**: `static $n = 0;` conserva el valor entre llamadas a la función. Debe inicializarse.
- **Variables de variables**: `$$var1` usa el valor de `$var1` como nombre de otra variable.

```
function contador() { static $n = 0; $n++; return $n; }   // 1, 2, 3 ... en cada llamada
$iva = 0.21;
function conIva($p) { global $iva; return $p * (1 + $iva); }

$var1 = "var2";  $var2 = "3";
echo $$var1;                          // 3
```

Variables superglobales (son arrays y se ven en cualquier sitio)

Variable	Contenido
`$_GET`	Parámetros de la URL (`?color=blue&sort=newest`).
`$_POST`	Datos enviados con un formulario method="post".
`$_REQUEST`	GET + POST + COOKIE juntos. Evítalo: no sabes de dónde vienen los datos.
`$_SERVER`	Información del servidor y la petición. `$_SERVER['REQUEST_METHOD']` es 'GET' o 'POST'.
`$_FILES`	Ficheros subidos con un formulario enctype="multipart/form-data".
`$_SESSION`, `$_COOKIE`	Variables de sesión (necesitan `session_start()`) y cookies.
`$_ENV`, `$GLOBALS`	Variables de entorno y todas las variables globales.

1.8 Constantes

```
const IVA = 0.21;                // fuera de clases y funciones
define('MAX_ENTRADAS', 4);       // equivalente; permite nombre dinámico
echo IVA, MAX_ENTRADAS;          // sin $
// Por convenio, en MAYÚSCULAS. Predefinidas: PHP_VERSION, PHP_EOL, PHP_INT_MAX, PHP_INT_SIZE
// Desde PHP 5.6 una constante puede contener un array: const PRECIOS = ['a' => 10, 'b' => 8];
```

1.9 Expresiones vacías: isset, empty, is_null

Valor	isset()	empty()	is_null()
variable no definida	false	true	(warning)
`null`	false	true	true
`""` (cadena vacía)	true	true	false
`0`, `"0"`, `false`, `[]`	true	true	false
`"abc"`, `5`	true	false	false

> **TRAMPA:** `empty("0")` es true: si un campo de formulario puede valer 0 (por ejemplo, nº de hijos), no uses `empty()` para ver si se rellenó. Compara con `=== ''`.

Un `echo` de una variable vacía, `null` o `false` se ve en blanco; `true` se ve como `1`.

1.10 Operadores

Tipo	Operadores
Aritméticos	`+ - * / % **` (potencia: `2**3` = 8). `intdiv(7,2)` = 3 (división entera).
Asignación	`= += -= *= /= %= .=` (`.=` añade texto a una cadena)
Cadenas	`.` concatenar
Incremento	`$a++` (usa y luego suma), `++$a` (suma y luego usa), `--`
Comparación	`==` igual valor, `!=` o `<>` distinto, `< <= > >=`, `<=>` (nave espacial: -1, 0, 1)
Idéntico	`===` mismo valor **y mismo tipo**, `!==`. Ej: `0 == "a"` ya no es true en PHP 8, pero `"1" == "01"` sí; `"1" === "01"` no.
Lógicos	`&&` / `and`, `||` / `or`, `!`, `xor` (uno u otro, no ambos). Usa `&&` y `||`: `and`/`or` tienen menos precedencia que `=`.
Fusión de null	`$v = $x ?? 'defecto';` equivale a `isset($x) ? $x : 'defecto'`. Imprescindible con `$_POST['campo'] ?? ''`.
Ternario	`$r = $a < $b ? 'menor' : 'mayor o igual';`

1.11 Estructuras de control

```
if ($a < $b) { echo "menor"; } elseif ($a > $b) { echo "mayor"; } else { echo "igual"; }

switch ($a) {                        // compara con == ; sin break "se cae" al siguiente case
    case 0:  echo "vale 0";  break;
    case 1:  echo "vale 1";  break;
    default: echo "ni 0 ni 1";
}

// match (PHP 8): compara con ===, devuelve valor, no necesita break, error si no hay caso
$tiempo = match ($servicio) {
    'tinte' => 120,
    'mechas' => 180,
    default => 30,
};
$nivel = match (true) { $nota < 5 => 'Suspenso', $nota < 7 => 'Aprobado', default => 'Notable o más' };

// Sintaxis alternativa: cómoda al mezclar con HTML
<?php if ($errores): ?>
   <ul><?php foreach ($errores as $e): ?><li><?= htmlspecialchars($e) ?></li><?php endforeach; ?></ul>
<?php endif; ?>
```

1.12 Bucles

```
for ($a = 1; $a <= 10; $a++) { echo $a; }
while ($a <= 10) { echo $a; $a++; }            // comprueba antes: puede no ejecutarse nunca
do { echo $a; $a++; } while ($a <= 10);        // comprueba después: se ejecuta al menos una vez

$datos = [1, 2, 3];
foreach ($datos as $valor) { echo "<br/>$valor"; }
$persona = ["Nombre" => "Ana", "Edad" => 20];
foreach ($persona as $clave => $valor) { echo "<br/>$clave: $valor"; }

break;      // sale del bucle                 break 2;    // sale de 2 bucles anidados
continue;   // pasa a la siguiente vuelta     continue 2; // sigue en el bucle exterior
```

2. Arrays

2.1 Definición y acceso

- **Escalares (indexados)**: la clave es un entero que empieza en 0.
- **Asociativos**: la clave es una cadena. Pueden mezclar tipos de valores. Son dinámicos (crecen solos).

```
$dias = array("Lunes", "Martes", "Miércoles");       // forma clásica
$meses = ["Enero", "Febrero", "Marzo"];               // forma corta (la habitual)
$vacio = [];                                          // array vacío
$info = ["nombre" => "Ana", "apellidos" => "Martín", "estatura" => 1.60];

echo $dias[1];                  // Martes
echo $info["nombre"];           // Ana
$dias[] = "Jueves";             // añade al final (clave 3)
$dias[10] = "Sábado";           // clave concreta
$info["telefono"] = "111222";   // añade clave nueva
unset($info["telefono"]);       // elimina un elemento

$matriz = [["Ana", 15], ["David", 19]];   // multidimensional
echo $matriz[1][0];                        // David
echo count($matriz);                       // 2 filas

var_dump($dias);  print_r($info);          // ver contenido (usa <pre> en el navegador)
```

2.2 Funciones de arrays

Función	Qué hace
`count($a)` / `sizeof($a)`	Número de elementos.
`is_array($v)`	true si es un array.
`array_keys($a)` / `array_values($a)`	Array con las claves / con los valores.
`array_key_exists($k, $a)`	true si existe la clave (aunque valga null). `isset($a[$k])` es false si vale null.
`in_array($v, $a, true)`	true/false si el valor está. El tercer parámetro `true` compara con === (recomendado).
`array_search($v, $a)`	Devuelve la **clave** del valor o `false`. Compara el resultado con `!== false` (la clave puede ser 0).
`array_merge($a, $b)`	Une arrays. Las claves numéricas se reindexan; las de texto repetidas se pisan.
`array_slice($a, ini, num)`	Extrae un trozo.
`array_unique($a)`	Quita valores repetidos.
`array_count_values($a)`	Cuántas veces aparece cada valor.
`array_sum($a)`, `max($a)`, `min($a)`	Suma, máximo y mínimo.
`array_map(fn, $a)`, `array_filter($a, fn)`	Transformar cada elemento / filtrar. Ej: `array_map(fn($x) => $x * 2, $a)`.
`array_diff($a, $b)`, `array_intersect($a, $b)`	Valores de $a que no están / sí están en $b.
`array_push($a, $x)` / `array_pop($a)`	Añade al final / quita y devuelve el último.
`array_unshift($a, $x)` / `array_shift($a)`	Añade al principio / quita y devuelve el primero.
`range(1, 10)`	Crea un array con una secuencia.
`sort($a)` / `rsort($a)`	Ordena de menor a mayor / de mayor a menor. **No mantiene índices.**
`asort($a)` / `arsort($a)`	Ordena por valor **manteniendo claves**.
`ksort($a)` / `krsort($a)`	Ordena por clave (útil en asociativos).
`shuffle($a)`	Desordena aleatoriamente.
`array_rand($a)`	Devuelve una clave al azar.
`implode(sep, $a)` / `explode(sep, $txt)`	Array a texto / texto a array.

> **OJO:** `sort`, `rsort`, `shuffle`, `array_push`... **modifican el propio array** y devuelven true/false; no escribas `$a = sort($a);`.

3. Funciones propias

- Se definen con `function nombre(parámetros) { ... }`. Hay que definirlas antes de llamarlas (salvo las definidas al nivel principal del fichero, que PHP carga primero).
- Devuelven un valor con `return`; si no hay return devuelven `null`.
- Los argumentos se pasan **por valor** (por defecto) o **por referencia** con `&$param` (la función modifica la variable original).
- Los parámetros opcionales llevan valor por defecto y van **al final**.
- PHP 8 permite indicar tipos: `function f(int $n, ?string $t = null): float`. Con `declare(strict_types=1);` al principio del fichero los tipos se comprueban estrictamente.

```
function precio_con_iva(float $precio, float $iva = 0.18): float {
    return $precio * (1 + $iva);
}
echo precio_con_iva(10);          // 11.8
echo precio_con_iva(10, 0.21);    // 12.1

function subir(float &$precio, float $iva = 0.18): void {   // por referencia
    $precio *= (1 + $iva);
}
$p = 10; subir($p);               // ahora $p vale 11.8

function datos(): array { return ['Ana', 19]; }
[$nombre, $edad] = datos();       // desestructurar el array devuelto

$doble = fn($x) => $x * 2;        // función flecha
```

Modularidad (se valora en el examen): separa la lógica en funciones pequeñas (`validar()`, `calcularTotal()`, `guardar()`, `leer()`) y deja el HTML lo más limpio posible. Comenta cada función con una línea que diga qué recibe y qué devuelve.

4. Funciones útiles de PHP

4.1 Tipos de datos

Función	Qué hace
`gettype($v)`	Devuelve el tipo como texto: array, boolean, double, integer, object, string, NULL...
`is_int`, `is_float`, `is_string`, `is_bool`, `is_array`, `is_null`, `is_object`	true si la variable es de ese tipo.
`is_numeric($v)`	true si es número o texto numérico (`"12.5"`). Con `"abc"` false. Ojo: `"1e3"` también es true.
`ctype_digit($s)`	true si el texto son solo dígitos 0-9 (sin signo ni punto).
`filter_var($v, FILTER_VALIDATE_INT)`	Devuelve el entero o `false`. Con opciones: `['options' => ['min_range' => 1, 'max_range' => 4]]`. Ojo: si el valor es 0 devuelve 0; compara con `=== false`.
`filter_var($e, FILTER_VALIDATE_EMAIL)`	Valida un email.
`isset($v)`, `empty($v)`, `unset($v)`	Está definida y no es null / está vacía / destruye la variable.

4.2 Fecha y hora

```
date_default_timezone_set('Europe/Madrid');    // poner al principio del fichero
echo date('d/m/Y H:i');            // 22/11/2024 18:30  (fecha y hora actual)
echo date('Y-m-d');                // 2024-11-22  (formato que usa <input type="date">)
$ts = strtotime("2022-07-11");     // texto -> timestamp (segundos desde 1970)
echo date('d/m/Y', $ts);           // timestamp -> texto
$hoy = time();                     // instante actual (int)
$info = getdate();                 // array: seconds, minutes, hours, mday, wday, mon, year, weekday, month...
checkdate(2, 30, 2024);            // false: comprueba que el día existe (mes, día, año)

// Con objetos (muy cómodo para validar fechas y cambiar de formato)
$f = DateTime::createFromFormat('Y-m-d', '2024-11-22');
echo $f->format('d/m/Y');          // 22/11/2024
$f && $f->format('Y-m-d') === '2024-11-22';   // true si la fecha es válida
```

Carácter	Resultado	Carácter	Resultado
`d`	día del mes, 2 dígitos (01-31)	`H`	hora 24h con 2 dígitos
`j`	día del mes sin ceros (1-31)	`h`	hora 12h con 2 dígitos
`N`	día de la semana (1 lunes ... 7 domingo)	`G` / `g`	hora 24h / 12h sin ceros
`w`	día de la semana (0 domingo ... 6 sábado)	`i`	minutos, 2 dígitos
`l` / `D`	nombre del día en inglés (Monday / Mon)	`s`	segundos, 2 dígitos
`m` / `n`	mes con 2 dígitos / sin ceros	`a` / `A`	am-pm / AM-PM
`F` / `M`	nombre del mes en inglés (January / Jan)	`t`	días que tiene el mes
`Y` / `y`	año de 4 / 2 dígitos	`L`	1 si el año es bisiesto
`W`	número de semana del año	`z`	día del año (0 = 1 de enero)

Los nombres de días y meses salen en inglés: para castellano hay que usar arrays (ver ejercicio 13).

4.3 Cadenas

Función	Qué hace
`strlen($s)`	Longitud en bytes (una `ñ` o `á` cuenta 2 en UTF-8). Con tildes usa `mb_strlen($s)`.
`strtolower` / `strtoupper` / `ucfirst` / `ucwords`	Minúsculas / mayúsculas / primera letra en mayúscula / cada palabra. Versión con tildes: `mb_strtolower($s, 'UTF-8')`.
`trim` / `ltrim` / `rtrim`	Quitan espacios (o los caracteres indicados) al inicio y final / solo inicio / solo final. `chop` es sinónimo de `rtrim`.
`str_replace(buscar, nuevo, $s)`	Reemplaza todas las apariciones. Admite arrays.
`substr($s, ini, len)`	Subcadena. `substr($s, -3)` = últimos 3 caracteres.

Función	Qué hace
`strpos($s, $b)` / `stripos`	Posición de la primera aparición o `false`. **Compara con `!== false`**, porque la posición 0 es válida.
`strrpos`, `strstr`, `strrchr`, `stristr`	Última posición / desde la primera aparición hasta el final / desde la última / ídem sin distinguir mayúsculas.
`str_contains`, `str_starts_with`, `str_ends_with`	PHP 8: devuelven true/false directamente.
`strcmp($a, $b)`, `strcasecmp`, `strncmp`, `strnatcmp`	Comparan: 0 si son iguales. `strcasecmp` ignora mayúsculas; `strnatcmp` orden natural (img2 < img10).
`str_pad($s, n, '0', STR_PAD_LEFT)`	Rellena hasta n caracteres.
`str_repeat($s, n)`	Repite el texto n veces.
`explode(';', $linea)`	Divide un texto por un separador y devuelve array. `explode(';', $l, 3)` limita las partes.
`implode('/', $array)`	Une los elementos con el separador. (`join` es sinónimo).
`str_split($s)`, `strrev($s)`, `wordwrap`	Trocea en caracteres / invierte / corta líneas.
`number_format($n, 2, ',', '.')`	Formatea número: `number_format(1234.5, 2, ',', '.')` -> `1.234,50`.
`htmlspecialchars($s, ENT_QUOTES, 'UTF-8')`	Convierte `< > & " '` en entidades HTML. **Obligatorio al mostrar datos del usuario.**
`nl2br($s)`	Convierte saltos de línea en ` ` (para textarea).
`strip_tags($s)`, `addslashes($s)`	Quita etiquetas HTML / escapa comillas.

4.4 Matemáticas y números aleatorios

```
rand(1, 10);  mt_rand(1, 10);  random_int(1, 10);   // entero aleatorio entre 1 y 10 (ambos incluidos)
round(3.14159, 2);   // 3.14        floor(4.7); // 4        ceil(4.2); // 5
abs(-3);  max(1, 5, 3);  min([4, 2, 9]);  sqrt(16);  pow(2, 3);  pi();  intdiv(7, 2);
```

5. Formularios web

5.1 Cómo llegan los datos a PHP

- `<form action="" method="post">`: `action` es el script que procesa (vacío o el propio nombre = el mismo fichero); `method` es `get` (datos en la URL, visibles, para consultas) o `post` (datos en el cuerpo, para enviar/guardar).
- Para subir ficheros: `method="post"` y `enctype="multipart/form-data"`.
- **El atributo `name` es la clave** en `$_POST`/`$_GET`; el `value` es el valor. Sin `name` el campo no se envía. El `id` solo sirve para el `<label for>` y CSS.
- Todo llega como **texto** (string), también los `type="number"` y `type="date"` (formato `YYYY-MM-DD`).
- Un botón submit con `name="enviar"` también viaja en `$_POST['enviar']`: sirve para saber si se pulsó.

Campo HTML	Qué llega en $_POST
`<input type="text" name="nombre">`	`$_POST['nombre']` siempre existe (string, puede ser vacío).
`<input type="radio" name="tipo" value="general">`	Solo llega si hay uno marcado. Si no, **la clave no existe** -> usa `$_POST['tipo'] ?? ''`.
`<input type="checkbox" name="cine" value="C">`	Solo llega si está marcado. Para comprobarlo: `isset($_POST['cine'])`.
`<input type="checkbox" name="aficiones[]" value="Cine">`	Varios checkbox con `[]`: `$_POST['aficiones']` es un array con los marcados (no existe si no hay ninguno).
`<select name="pais">`	Un string con el `value` de la opción elegida.
`<select name="descuentos[]" multiple>`	Un **array** con las opciones seleccionadas (no existe si no se selecciona ninguna). Con Ctrl+clic se eligen varias.
`<textarea name="comentario">`	String (puede incluir saltos de línea).
`<input type="file" name="foto">`	No va en `$_POST`, va en `$_FILES['foto']`.
`<input type="hidden" name="id" value="5">`	Dato que viaja sin verse (el usuario puede modificarlo: valídalo).

5.2 Patrón básico: el mismo fichero muestra el formulario y lo procesa

```
<?php
$nombre = '';                                   // 1) valores por defecto (primera carga)
$errores = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {    // 2) ¿se ha enviado el formulario?
    $nombre = trim($_POST['nombre'] ?? '');     //    recoger con ?? para evitar "undefined array key"
    if ($nombre === '') { $errores[] = 'El nombre es obligatorio.'; }
    if (!$errores) {
        // 3) todo correcto: procesar (guardar, mostrar resultado...)
    }
}
?>
<form action="" method="post">
  <input type="text" name="nombre" value="<?= htmlspecialchars($nombre) ?>">
  <input type="submit" name="enviar" value="Enviar">
</form>
```

- Para saber si se envió: `$_SERVER['REQUEST_METHOD'] === 'POST'` o `isset($_POST['enviar'])` (con corchetes, no paréntesis).
- El orden importa: primero **procesar y validar**, después **pintar** el formulario con los datos recordados.

5.3 Recordar los datos en el formulario

Al recargar la página, los campos se rellenan con lo que el usuario escribió usando `value`, `checked` y `selected`:

```
// Texto
<input type="text" name="nombre" value="<?= htmlspecialchars($nombre) ?>">

// Radio: marcar el que coincide
<input type="radio" name="tipo" value="general" <?= $tipo === 'general' ? 'checked' : '' ?>> General
<input type="radio" name="tipo" value="mayor60"  <?= $tipo === 'mayor60'  ? 'checked' : '' ?>> Mayor de 60

// Select generado desde un array
<select name="espectaculo">
<?php foreach ($espectaculos as $e): ?>
  <option value="<?= $e ?>" <?= $espectaculo === $e ? 'selected' : '' ?>><?= $e ?></option>
```

```
<?php endforeach; ?>
</select>

// Checkbox / select múltiple (array): in_array para saber si estaba marcado
<input type="checkbox" name="aficiones[]" value="Cine" <?= in_array('Cine', $aficiones) ? 'checked' : '' ?>>

// Fecha por defecto = hoy
<input type="date" name="fecha" value="<?= $fecha ?>">     // $fecha = date('Y-m-d') la primera vez
```

OJO: `htmlspecialchars` también en `value="..."`: si el usuario escribe unas comillas, sin escapar rompería el HTML (y permitiría XSS).

5.4 Validación en el servidor

- Se valida **siempre en el servidor**: la validación del navegador (`required`, `min`, `max`, JavaScript) se puede saltar (Man in the Middle, peticiones manuales).
- Hay que protegerse de **XSS** (Cross-Site Scripting): un usuario puede escribir `<script>alert('Hack')</script>`; al mostrarlo sin escapar, el navegador lo ejecuta. Solución: `htmlspecialchars` al mostrar.
- Si hay errores: se vuelve a mostrar el formulario con los datos correctos rellenos, y se indica cada error y su motivo.

```
function validar(array $d): array {
    $errores = [];
    if ($d['nombre'] === '')                        $errores[] = 'El nombre es obligatorio.';
    if ($d['edad'] === '')                          $errores[] = 'La edad es obligatoria.';
    elseif (filter_var($d['edad'], FILTER_VALIDATE_INT) === false)
                                                    $errores[] = 'La edad debe ser un número entero.';
    elseif ($d['edad'] < 0 || $d['edad'] > 120)     $errores[] = 'La edad debe estar entre 0 y 120.';
    if (!preg_match('/^[\w.+-]+@[\w-]+(\.[\w-]+)+$/', $d['email']))
                                                    $errores[] = 'El email no es válido.';
    $f = DateTime::createFromFormat('Y-m-d', $d['fecha']);
    if (!$f || $f->format('Y-m-d') !== $d['fecha'])  $errores[] = 'La fecha no es válida.';
    return $errores;
}
```

Quiero comprobar...	Cómo
Campo obligatorio (texto)	`trim($_POST['x'] ?? '') === ''` (el `trim` evita que valga ' ').
Obligatorio y puede ser 0	`$v === ''` (no uses `empty`).
Número entero	`filter_var($v, FILTER_VALIDATE_INT) !== false` o `ctype_digit($v)`.
Número entre 1 y 4	Primero que sea entero, luego `$n >= 1 && $n <= 4`.
Valor dentro de una lista	`in_array($v, $permitidos, true)`.
Selección múltiple con algún valor	`!empty($_POST['x'])` o `isset($_POST['x'])`.
Fecha válida	`DateTime::createFromFormat` + comparar el `format` / `checkdate()`.
Dos opciones incompatibles	`$tipo === 'mayor60' && in_array('Familia Numerosa', $descuentos, true)`.
Formato (email, DNI, teléfono)	`preg_match()` con expresión regular (apartado 8).

5.5 Paso de datos por URL (GET)

Los parámetros van tras `?` y se separan con `&`: `pagina.php?nombre=David&edad=19`. En PHP se leen en `$_GET`. Se usa en enlaces y en formularios con `method="get"`.

```
// ejercicio4.php
$personas = ['Ana' => 15, 'David' => 19, 'Andrés' => 23];
echo '<ul>';
foreach ($personas as $nombre => $edad) {
    echo '<li><a href="ejercicio4Detalle.php?nombre=' . urlencode($nombre) . '&edad=' . $edad . '">'
         . htmlspecialchars($nombre) . '</a>: ' . $edad . '</li>';        // urlencode: tildes, espacios
}
echo '</ul>';

// ejercicio4Detalle.php
$nombre = $_GET['nombre'] ?? null;
$edad   = $_GET['edad'] ?? null;
if ($nombre === null || $edad === null || !ctype_digit($edad)) { exit('Datos no válidos'); }
echo 'Nombre: ' . htmlspecialchars($nombre) . '<br/>Edad: ' . (int)$edad;
```

5.6 Subir ficheros

- Formulario con `enctype="multipart/form-data"` y `<input type="file" name="foto">`.
- `$_FILES['foto']` es un array con: `name` (nombre original), `type` (MIME declarado por el navegador), `size` (bytes), `tmp_name` (ruta temporal en el servidor) y `error` (código).
- El fichero temporal se mueve a su destino con `move_uploaded_file()`; si no se mueve, PHP lo borra al acabar.
- La carpeta destino (por ejemplo `fotos/`) debe existir: créala a mano o con `mkdir`.

```
if (isset($_FILES['foto']) && $_FILES['foto']['error'] === UPLOAD_ERR_OK) {
    $carpeta = 'fotos/';
    if (!is_dir($carpeta) && !mkdir($carpeta, 0755, true)) {
        throw new RuntimeException('No se pudo crear la carpeta fotos');
    }
    $permitidos = ['image/jpeg', 'image/png', 'image/gif'];
    if (!in_array(mime_content_type($_FILES['foto']['tmp_name']), $permitidos, true)) {
        $errores[] = 'La foto debe ser JPG, PNG o GIF.';
    } elseif ($_FILES['foto']['size'] > 2 * 1024 * 1024) {
        $errores[] = 'La foto no puede superar 2 MB.';
    } else {
        $destino = $carpeta . basename($_FILES['foto']['name']);     // basename: evita rutas maliciosas
        if (move_uploaded_file($_FILES['foto']['tmp_name'], $destino)) {
            echo 'Fichero subido con éxito<br/><img src="' . htmlspecialchars($destino) . '" width="120">';
        } else {
            $errores[] = 'Error al subir la foto.';
        }
    }
} elseif (($_FILES['foto']['error'] ?? UPLOAD_ERR_NO_FILE) === UPLOAD_ERR_NO_FILE) {
    // el usuario no eligió ningún fichero (puede ser correcto o no según el enunciado)
}
```

Códigos de `error`: `UPLOAD_ERR_OK` (0) todo bien, `UPLOAD_ERR_NO_FILE` (4) no se subió nada, `UPLOAD_ERR_INI_SIZE` (1) supera `upload_max_filesize` del php.ini, `UPLOAD_ERR_PARTIAL` (3) subida incompleta.

5.7 Formulario en varios pasos (ejercicio 16)

Idea: un único fichero con varios botones submit con distinto `name`; según el botón pulsado se pinta un trozo u otro. Se conserva el nº de alumnos con el propio campo.

```
$num = (int)($_POST['num'] ?? 0);
$alumnos = $_POST['alumnos'] ?? [];
?>
<form method="post" action="">
  Nº Alumnos <input type="number" name="num" value="<?= $num ?>">
  <input type="submit" name="crear" value="Crear">
<?php if (isset($_POST['crear']) || isset($_POST['mostrar'])): ?>
  <?php for ($i = 0; $i < $num; $i++): ?>
    <br/>Alumno <?= $i + 1 ?> <input type="text" name="alumnos[]" value="<?= htmlspecialchars($alumnos[$i] ??
        '') ?>">
  <?php endfor; ?>
  <br/><input type="submit" name="mostrar" value="Mostrar">
<?php endif; ?>
</form>
<?php if (isset($_POST['mostrar'])): ?>
  <p>Alumnos:</p><ul><?php foreach ($alumnos as $a): ?><li><?= htmlspecialchars($a) ?></li><?php endforeach;
      ?></ul>
<?php endif; ?>
```

5.8 Redirección

```
header('Location: pagina.php');   // debe ejecutarse ANTES de enviar cualquier salida (echo, HTML, espacios)
exit;                              // siempre exit después, si no el script sigue ejecutándose
```

Patrón útil (Post-Redirect-Get): tras guardar un POST correctamente, redirige a la misma página para que, al refrescar (F5), el navegador no reenvíe el formulario y duplique el registro. Ojo: si recuerdas datos en el formulario, la redirección los pierde; en el examen lee bien qué pide.

6. Ficheros de texto

6.1 Modos de apertura de fopen

Modo	Descripción
`r`	Solo lectura; puntero al principio. Si no existe, falla (false).
`r+`	Lectura y escritura; puntero al principio.
`w`	Solo escritura; **vacía el fichero** (lo trunca a 0). Si no existe, lo crea.
`w+`	Lectura y escritura; también lo vacía.
`a`	Solo escritura; puntero al **final** (añade sin borrar). Si no existe, lo crea.
`a+`	Lectura y escritura; escribe siempre al final.
`x` / `x+`	Solo crea: si el fichero ya existe, falla con warning.

6.2 Funciones

Función	Qué hace
`$f = fopen($nombre, 'a')`	Abre y devuelve el descriptor (o `false` si falla).
`fclose($f)`	Cierra el fichero.
`fgets($f)`	Lee una línea (incluye el salto de línea). `false` al final.
`feof($f)`	true si el puntero está al final.
`fwrite($f, $txt)` / `fputs`	Escribe texto. Para fin de línea añade `PHP_EOL`.
`rewind($f)`, `fseek($f, n)`, `ftell($f)`	Puntero al principio / mover el puntero / posición actual.
`file($nombre, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES)`	Lee todo el fichero a un array (una línea por elemento, sin saltos ni líneas vacías).
`file_get_contents($nombre)`	Lee todo el fichero en un string.
`file_put_contents($nombre, $txt, FILE_APPEND | LOCK_EX)`	Escribe (o añade con FILE_APPEND) de una sola vez. Devuelve nº de bytes o `false`.
`readfile($nombre)`	Lee y muestra el contenido; devuelve nº de bytes.
`file_exists`, `is_file`, `is_dir`, `is_readable`, `is_writable`	Comprobaciones (existe, es fichero, es carpeta, se puede leer, se puede escribir).
`filesize($f)`, `filemtime($f)`	Tamaño en bytes / fecha de última modificación (timestamp).
`copy(origen, destino)`, `rename(a, b)`, `unlink($f)`	Copiar / renombrar o mover / borrar.
`mkdir($d, 0755, true)`, `rmdir($d)`	Crear carpeta (true = crea las intermedias) / borrar carpeta vacía.
`opendir`, `readdir`, `closedir`, `scandir($d)`	Recorrer una carpeta (`scandir` devuelve un array con los nombres).
`getcwd()`, `chdir($d)`	Directorio de trabajo actual / cambiarlo.

6.3 Patrón: guardar registros separados por ;

```
const FICHERO = 'datos.txt';

// Guardar una línea:  Ana;15;Corte señora
function guardarLinea(string $linea): void {
    $ok = file_put_contents(FICHERO, $linea . PHP_EOL, FILE_APPEND | LOCK_EX);
    if ($ok === false) {
        throw new RuntimeException('No se pudo escribir en ' . FICHERO);
    }
}

// Leer todas las líneas y convertirlas en arrays
function leerLineas(): array {
    if (!file_exists(FICHERO)) { return []; }                    // fichero aún no creado: lista vacía
    $lineas = file(FICHERO, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    if ($lineas === false) { throw new RuntimeException('No se pudo leer ' . FICHERO); }
    return array_map(fn($l) => explode(';', $l), $lineas);
}

// Mostrar siempre una tabla (aunque esté vacía)
```

```
echo '<table border="1"><tr><th>Nombre</th><th>Edad</th></tr>';
foreach (leerLineas() as $c) {
    echo '<tr><td>' . htmlspecialchars($c[0]) . '</td><td>' . htmlspecialchars($c[1]) . '</td></tr>';
}
echo '</table>';
```

OJO: Si el usuario escribe un `;` o un salto de línea en un campo, rompe el formato del fichero. Límpialo antes de guardar: `str_replace([';', "\r", "\n"], ' ', $texto)`.

OTRA FORMA: Recorrido clásico con fopen: `$f = fopen($n, 'r'); while (($l = fgets($f)) !== false) { ... } fclose($f);` (comprueba que `$f` no sea false).

7. Control de excepciones

- `Throwable` es la interfaz raíz. De ella derivan **`Error`** (fallos del motor: `TypeError`, `DivisionByZeroError`, `ParseError`, `ArgumentCountError`, `ValueError`) y **`Exception`** (las que lanzamos nosotros: `RuntimeException`, `InvalidArgumentException`, `LogicException`, `PDOException`...).
- `catch (Throwable $e)` captura las dos ramas. Se pueden poner varios `catch` de más específico a más general.
- `finally` se ejecuta siempre (haya error o no); útil para cerrar ficheros.
- Los **warnings** de PHP (por ejemplo fallo de `fopen`) no son excepciones: se detectan comprobando el valor devuelto (`false`) y lanzando tú la excepción.
- PHP 8: dividir entre 0 con `/` o `%` lanza `DivisionByZeroError`. `fdiv()` devuelve INF en vez de lanzar.

```
try {
    $f = fopen('entradas.txt', 'a');
    if ($f === false) { throw new RuntimeException('No se puede abrir el fichero de entradas'); }
    fwrite($f, $linea . PHP_EOL);
} catch (RuntimeException $e) {
    echo '<p class="error">Error al guardar: ' . htmlspecialchars($e->getMessage()) . '</p>';
} catch (Throwable $e) {                      // cualquier otra cosa
    echo '<p class="error">Error inesperado.</p>';
    error_log($e->getMessage());              // al log del servidor, no al usuario
} finally {
    if (isset($f) && $f) { fclose($f); }
}

// Excepción propia
class EntradaInvalidaException extends InvalidArgumentException {}
throw new EntradaInvalidaException('El número de entradas debe estar entre 1 y 4');

// $e->getMessage()   $e->getCode()   $e->getLine()   $e->getFile()
```

> **EXAMEN:** En el examen se puntúa el control de situaciones excepcionales: fichero que no existe o no se puede escribir, división por 0, datos que no llegan en `$_POST`, línea del fichero mal formada, carpeta que no existe, etc. Los mensajes al usuario deben ser claros (no le enseñes rutas internas ni el stack trace).

8. Programación orientada a objetos

8.1 Clase, objeto, constructor y destructor

```
// empleado.php  (se carga con include_once / require_once 'empleado.php';)
class Empleado {
    public string $nombre;              // accesible desde fuera
    private float $sueldo;              // solo desde dentro de la clase
    protected string $puesto = 'Sin asignar';   // desde la clase y sus hijas
    const MAXIMO = 5000;                // constante de clase: Empleado::MAXIMO
    private static int $total = 0;      // pertenece a la clase, no al objeto

    public function __construct(string $nombre, float $sueldo = 1000) {
        $this->nombre = $nombre;        // $this = el objeto actual
        $this->sueldo = $sueldo;
        self::$total++;                 // static: self:: o Empleado::
    }
    public function __destruct() { self::$total--; }     // al hacer unset($obj) o terminar el script

    public function mostrar(): void {
        echo "Nombre: {$this->nombre}<br/>Sueldo: {$this->sueldo}<br/>";
    }
    public function getSueldo(): float { return $this->sueldo; }            // getter
    public function setSueldo(float $s): void {                              // setter con validación
        if ($s < 0 || $s > self::MAXIMO) { throw new InvalidArgumentException('Sueldo no válido'); }
        $this->sueldo = $s;
    }
    public static function cuantos(): int { return self::$total; }          // método estático
    public function __toString(): string { return $this->nombre; }          // echo $obj
}

include_once 'empleado.php';
$conserje = new Empleado('Andrés');        // crear objeto
$conserje->nombre = 'José';                // atributo público
$conserje->mostrar();                      // método
echo Empleado::cuantos(), Empleado::MAXIMO;   // estático y constante: Clase::
unset($conserje);                          // llama al destructor
```

PHP 8 - promoción de propiedades: se declaran las propiedades directamente en el constructor, ahorrando código:

```
class Cita {
    public function __construct(
        private string $fecha,
        private string $hora,
        private string $cliente,
        private string $servicio
    ) {}
    public function getCliente(): string { return $this->cliente; }
}
```

Visibilidad	Accesible desde...
`public` (o `var` en PHP antiguo)	Cualquier sitio.
`private`	Solo dentro de la propia clase.
`protected`	La clase y sus descendientes (hijas).

8.2 Herencia, clases abstractas, finales e interfaces

```
class Gerente extends Empleado {                    // hereda atributos y métodos (no hay herencia múltiple)
    public function __construct(string $nombre, private int $personasACargo = 0) {
        parent::__construct($nombre, 3000);         // el constructor del padre NO se llama solo
    }
    public function mostrar(): void {               // sobrescribir un método
        parent::mostrar();
        echo "Personas a cargo: {$this->personasACargo}<br/>";
    }
}

abstract class Figura {                              // no se puede instanciar con new
    abstract public function area(): float;          // sin cuerpo: las hijas DEBEN implementarlo
    public function describir(): string { return 'Área: ' . $this->area(); }
}
class Circulo extends Figura {
    public function __construct(private float $r) {}
    public function area(): float { return pi() * $this->r ** 2; }
}
```

```php
final class Util {}                                 // no puede tener hijas
class A { final public function f() {} }            // método que no se puede sobrescribir

interface Guardable {                               // solo declara métodos (sin atributos)
    public function toLinea(): string;
}
class Entrada implements Guardable, Stringable {    // se pueden implementar varias
    public function toLinea(): string { return 'datos'; }
    public function __toString(): string { return $this->toLinea(); }
}
// instanceof: $obj instanceof Empleado   -> true/false
```

8.3 Namespaces

Organizan el código y evitan colisiones de nombres (parecido a los paquetes de Java).

```php
<?php // Modelos/Usuario.php
namespace App\Modelos;
class Usuario { public function saludar() { return "Hola desde la clase Usuario."; } }

<?php // index.php
require 'Modelos/Usuario.php';        // el namespace NO carga el fichero: hay que incluirlo
use App\Modelos\Usuario;              // importar la clase
$u = new Usuario();                   // o: new \App\Modelos\Usuario();
echo $u->saludar();
// Dentro de un namespace, las clases/funciones globales de PHP se escriben con \ delante: new \DateTime()
```

8.4 include / require

Sentencia	Si el fichero no existe
`include 'f.php'`	Warning: el script continúa.
`require 'f.php'`	Error fatal: el script se detiene (úsalo para clases y funciones imprescindibles).
`include_once` / `require_once`	Igual, pero si ya se incluyó antes no lo vuelve a cargar (evita 'cannot redeclare').

Las rutas son relativas al script que se ejecuta. Para ser seguro: `require_once __DIR__ . '/Entrada.php';`.

9. Expresiones regulares

Toda expresión regular de PHP va **entre delimitadores** (normalmente `/`; también `#`, `~`, `%`). Los modificadores van después del delimitador de cierre: `/patrón/i`.

Modificador	Significado
`i`	No distingue mayúsculas de minúsculas.
`m`	`^` y `$` coinciden con inicio y fin de cada línea.
`s`	El punto `.` también coincide con saltos de línea.
`u`	Trata el texto como UTF-8 (necesario para tildes y `\p{L}`).

Metacarácter	Significado	Cuantificador	Significado
`.`	Cualquier carácter	`*`	0 o más veces
`\d` / `\D`	Dígito / no dígito	`+`	1 o más veces
`\w` / `\W`	Letra, número o _ / lo contrario	`?`	0 o 1 vez (opcional)
`\s` / `\S`	Espacio en blanco / no espacio	`{n}`	Exactamente n veces
`^ $`	Inicio / fin de cadena	`{n,}`	n o más veces
`\b`	Límite de palabra	`{n,m}`	Entre n y m veces
`[abc] [a-z] [^0-9]`	Uno de estos / rango / cualquiera menos estos	`(ab|cd)`	Alternativa y grupo (se captura)

```
preg_match($patron, $texto, $coincidencias);   // 1 si coincide, 0 si no, false si el patrón es inválido
                                               // $coincidencias[0] = lo encontrado; [1], [2]... = grupos ( )
preg_match_all($patron, $texto, $todas);       // todas las coincidencias
preg_replace('/\s+/', ' ', $texto);            // reemplazar con patrón
preg_split('/[,;]\s*/', $texto);               // dividir con patrón

// Patrones típicos de validación (con ^ y $ para exigir que TODO el texto cumpla el patrón)
'/^[\w.+-]+@[\w-]+(\.[\w-]+)+$/'      // email sencillo
'/^\d{8}[A-Za-z]$/'                   // DNI: 8 dígitos y una letra
'/^[6789]\d{8}$/'                     // teléfono español de 9 dígitos
'/^\d{5}$/'                           // código postal
'#^\d{2}/\d{2}/\d{4}$#'               // fecha dd/mm/aaaa (con # no hay que escapar la /)
'/^[\p{L} ]+$/u'                      // solo letras (con tildes y ñ) y espacios
'/^(?=.*[A-Z])(?=.*\d).{8,}$/'        // contraseña: 8+ caracteres, una mayúscula y un número
```

OJO: Sin `^` y `$` el patrón coincide con una **parte** del texto (el ejemplo de las diapositivas encuentra un correo dentro de una frase). Para validar un campo, añade siempre ambos.

10. Ejercicios de la unidad resueltos

Soluciones cortas para consultar. Todas son ficheros `.php` independientes salvo que se indique lo contrario.

Ejercicios 1 y 2 - variables y tipos

```
<?php
$nombreCompleto = "Ana Corredor Pérez";      // texto literal
$edad = 23;  $poblacion = "Navalmoral";
echo "<p>$nombreCompleto</p>";
echo "<p>Edad: $edad - Población: $poblacion</p>";
echo "<ul>";                                  // ejercicio 2: lista con el tipo de cada variable
foreach (['nombreCompleto' => $nombreCompleto, 'edad' => $edad, 'poblacion' => $poblacion] as $n => $v) {
    echo "<li>\$$n: " . gettype($v) . "</li>";
}
echo "</ul>";
```

Ejercicio 3 - nota y calificación

```
$nota = "8,5";                                          // llega con coma, como en el enunciado
$n = (float)str_replace(',', '.', $nota);               // PHP solo entiende el punto decimal
$cal = match (true) {
    $n < 5 => 'Suspenso', $n < 6 => 'Suficiente', $n < 7 => 'Bien',
    $n < 9 => 'Notable',  default => 'Sobresaliente',
};
echo "Nota: $nota<br/>Calificación: $cal";
```

Ejercicio 5 - tablas de multiplicar con filas alternas

```
echo '<table border="1">';
for ($i = 1; $i <= 10; $i++) {
    $color = ($i % 2 === 1) ? '#999999' : '#ffffff';
    echo "<tr style=\"background:$color\">";
    for ($j = 1; $j <= 10; $j++) { echo '<td>' . ($i * $j) . '</td>'; }
    echo '</tr>';
}
echo '</table>';
```

Ejercicio 6 - operaciones con dos enteros aleatorios distintos

```
$a = rand(0, 10);
do { $b = rand(0, 10); } while ($b === $a);           // distintos
try {
    if (!is_int($a) || !is_int($b)) { throw new InvalidArgumentException('Los valores deben ser enteros'); }
    echo "La suma de $a+$b es: " . ($a + $b) . '<br/>';
    echo "La resta de $a-$b es: " . ($a - $b) . '<br/>';
    echo "La multiplicación de $a*$b es: " . ($a * $b) . '<br/>';
    if ($b === 0) { throw new DivisionByZeroError('No se puede dividir ni calcular el resto entre 0'); }
    echo "La división de $a/$b es: " . ($a / $b) . '<br/>';
    echo "El resto de $a%$b es: " . ($a % $b) . '<br/>';
    echo "La potencia de $a<sup>$b</sup> es: " . ($a ** $b);
} catch (Throwable $e) {
    echo '<p style="color:red">Error: ' . htmlspecialchars($e->getMessage()) . '</p>';
}
```

Ejercicio 7 - tres números ordenados y repetidos

```
$n = [rand(1, 10), rand(1, 10), rand(1, 10)];
echo 'Los números generados son: ' . implode(', ', $n) . '<br/>';
foreach (array_count_values($n) as $num => $veces) {
    if ($veces > 1) { echo "El número $num está repetido<br/>"; }
}
sort($n);
echo 'Los números ordenados son: ' . implode(', ', $n);
```

Ejercicio 8 - repetir mientras sea menor que 50

```
do { $n = rand(0, 100); echo "$n<br/>"; } while ($n < 50);
```

Ejercicios 9, 10, 11 y 12 - arrays

```
$planetas = ['Mercurio', 'Venus', 'Tierra', 'Marte', 'Júpiter', 'Saturno', 'Urano', 'Neptuno'];  // 9
foreach ($planetas as $i => $p) { echo ($i + 1) . ". $p<br/>"; }

$serie = [];                                    // 11: "serie x2" = 2, 4, 8, ... (si fueran cuadrados: $i **
    2)
for ($i = 1; $i <= 10; $i++) { $serie[] = 2 ** $i; }
```

```php
echo implode(', ', $serie);

function pintar(array $a, string $titulo): void {                              // 10
    echo "$titulo - Número de elementos " . sizeof($a);
    echo '<table border="1" style="background:#add8e6"><tr>';
    foreach (array_keys($a) as $k) { echo "<td>$k</td>"; }
    echo '</tr><tr>';
    foreach ($a as $v) { echo '<td>' . htmlspecialchars((string)$v) . '</td>'; }
    echo '</tr></table><br/>';
}
pintar(['Cougar', 'Ford', '', 2500, 'V6', 182], '$array1: Array Escalar');
pintar(['Modelo' => 'Cougar', 'Marca' => 'Ford', 'fecha' => '', 'CC' => 2500,
        'Motor' => 'V6', 'Potencia' => 182], '$array2: Array Asociativo');

$datos = [2345.65, 'Carlos', 34, ['Nombre' => 'María', 'Edad' => 19], true];    // 12
foreach ($datos as $i => $v) {
    echo "Posición $i: " . gettype($v) . ' - Valor: ';
    if (is_array($v)) { foreach ($v as $k => $x) { echo "<br/>=> $k: $x"; } }
    else { echo $v; }                              // true se ve como 1
    echo '<br/>';
}
```

Ejercicio 13 - fecha en castellano: Lunes, 28 de Septiembre de 2023

```php
date_default_timezone_set('Europe/Madrid');
$dias  = ['Domingo', 'Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado'];   // índice = date('w')
$meses = [1 => 'Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo', 'Junio', 'Julio',
          'Agosto', 'Septiembre', 'Octubre', 'Noviembre', 'Diciembre'];               // índice = date('n')
echo $dias[date('w')] . ', ' . date('j') . ' de ' . $meses[(int)date('n')] . ' de ' . date('Y');
```

Ejercicios 14 y 15 - formulario con todos los tipos de campo y foto

Fichero 1: formulario (`method="post"`, `enctype="multipart/form-data"`) con: text (nombre, apellidos), password, radio (`sexo`: H/M), date (`fechaN`), `select multiple name="pais[]"`, select simple `hijos` con 0-4, file `foto`, checkbox `aficiones[]` (Cine, Deporte, Literatura) y textarea. Fichero 2: recorrer los datos.

```php
// tratar.php
$obligatorios = ['nombre' => 'Nombre', 'apellidos' => 'Apellidos', 'fechaN' => 'Fecha de nacimiento'];
$errores = [];
foreach ($obligatorios as $campo => $texto) {                 // ejercicio 15: campos siempre rellenos
    if (trim($_POST[$campo] ?? '') === '') { $errores[] = "El campo $texto es obligatorio."; }
}
if ($errores) {
    foreach ($errores as $e) { echo '<p style="color:red">' . htmlspecialchars($e) . '</p>'; }
    echo '<a href="formulario.php">Volver al formulario</a>';
    exit;
}
echo '<h2>Datos seleccionados</h2>';
foreach ($_POST as $clave => $valor) {
    if (is_array($valor)) { $valor = implode(' ', $valor); }          // pais[] y aficiones[]
    echo 'Elemento: ' . htmlspecialchars($clave) . ' => ' . htmlspecialchars($valor) . '<br/>';
}
// la foto: ver apartado 5.6 (move_uploaded_file a la carpeta fotos/ y <img>)
echo '<a href="formulario.php">Volver al formulario</a>';
```

Ejercicio 17 - gestión de citas (clase Cita + fichero)

```php
// Cita.php
class Cita {
    public function __construct(private string $fecha, private string $hora,
                                private string $cliente, private string $servicio) {}
    public function getTiempo(): int {                          // NO es un atributo: se calcula
        return match ($this->servicio) {
            'Corte señora' => 30, 'Corte caballero' => 15, 'Tinte' => 120, 'Mechas' => 180,
            default => throw new UnexpectedValueException("Servicio desconocido: {$this->servicio}"),
        };
    }
    public function toLinea(): string {
        return implode(';', [$this->fecha, $this->hora, $this->cliente, $this->servicio]);
    }
    public static function desdeLinea(string $l): Cita {
        $c = explode(';', $l);
        if (count($c) !== 4) { throw new UnexpectedValueException("Línea incorrecta: $l"); }
        return new Cita(...$c);                                  // ... reparte el array en parámetros
    }
    public function getFecha(): string { return $this->fecha; }
    public function getHora(): string { return $this->hora; }
    public function getCliente(): string { return $this->cliente; }
    public function getServicio(): string { return $this->servicio; }
}
// Después: formulario (date, time, text, select con los 4 servicios) -> validar -> new Cita(...) ->
```

```
// file_put_contents('citas.txt', $cita->toLinea() . PHP_EOL, FILE_APPEND) -> leer con file() y mostrar
// una tabla con $c->getTiempo() . " min".   (misma estructura que el examen modelo, apartado 11)
```

Ejercicio 18 - viviendas

Misma estructura que el ejercicio 17 y que el examen: clase `Vivienda` con tipo, zona, dirección, nº de habitaciones, precio, tamaño, extras y observaciones. Los extras llegan como array (`name="extras[]"`, con `value="Garaje"`...) y se guardan con `implode('/', $extras)`. Valida que todo lo demás no esté vacío, que precio y tamaño sean números positivos y que tipo/zona/habitaciones pertenezcan a las opciones permitidas. Guarda `tipo;zona;dirección;habitaciones;precio;tamaño;extras;observaciones` (limpiando los `;` y saltos de línea de las observaciones).

Ejercicio 19 - menú configurable con POO

```
class Menu {
    public function __construct(private array $opciones, private string $colorFondo, private string
        $colorTexto) {}
    public function pintar(): void {
        foreach ($this->opciones as $op) {
            echo '<span style="background:' . htmlspecialchars($this->colorFondo) . ';color:'
               . htmlspecialchars($this->colorTexto) . ';padding:2px 6px">' . htmlspecialchars($op) .
                   '</span> ';
        }
    }
}
// Paso 1: form con "Número de opciones" (post).  Paso 2: for ($i...) { <input name="opciones[]"> },
// <input type="color" name="fondo">, <input type="color" name="texto">.
// Paso 3: $menu = new Menu(array_filter($_POST['opciones'], fn($o) => trim($o) !== ''), $_POST['fondo'],
    $_POST['texto']);
//         $menu->pintar();
```

Juego de capitales (informacion.txt, formato número;país;capital)

```
<?php
$fichero = 'informacion.txt';
$lineas = file($fichero, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
if ($lineas === false || !$lineas) { exit('No se pudo leer informacion.txt'); }

$paises = [];
foreach ($lineas as $l) {
    $partes = explode(';', $l);
    if (count($partes) < 3) { continue; }                          // ignora líneas mal formadas
    $paises[(int)$partes[0]] = ['pais' => trim($partes[1]), 'capital' => trim($partes[2])];
}

$mensaje = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {                       // comprobar la respuesta anterior
    $id = (int)($_POST['id'] ?? 0);
    $resp = trim($_POST['respuesta'] ?? '');
    if (isset($paises[$id])) {
        $ok = mb_strtolower($resp, 'UTF-8') === mb_strtolower($paises[$id]['capital'], 'UTF-8');
        $mensaje = $ok ? '¡Correcto!' : "Incorrecto: la capital de {$paises[$id]['pais']} es
            {$paises[$id]['capital']}.";
    }
}
$id = array_rand($paises);                                         // nueva pregunta al azar
?>
<p><?= htmlspecialchars($mensaje) ?></p>
<form method="post" action="">
  ¿Cuál es la capital de <b><?= htmlspecialchars($paises[$id]['pais']) ?></b>?
  <input type="hidden" name="id" value="<?= $id ?>">      <!-- el id viaja, la solución no -->
  <input type="text" name="respuesta" autofocus>
  <input type="submit" value="Comprobar">
</form>
```

> **OJO:** Si guardas `informacion.txt` en ANSI y PHP espera UTF-8 (o al revés) las tildes salen mal (`Bogot■`). Guarda el fichero en **UTF-8 sin BOM**. Un BOM al principio hace que el primer número no se convierta bien.

11. Examen modelo resuelto: venta de entradas

Enunciado (Recuperación T2, 12/11/2024, 3 horas): carpeta `2425RECUPT2` con todos los ficheros; al acabar, comprimirla y subirla a Classroom. En todos los ejercicios se valora el **control de excepciones, la modularidad, la claridad de los mensajes y los comentarios**. Puntuación: ejercicio 1 = 3, ejercicio 2 = 3, ejercicio 3 = 4.

Ej.	Qué pide
1 (3 p)	`Ejercicio1.php`: formulario 'Venta de Entradas' (nombre completo, tipo de entrada con radio General/Mayor de 60/Menor de 6 años, espectáculo select, fecha del evento rellenada por PHP con la fecha actual, número de entradas, descuentos en select múltiple tratado como **array**, botón Comprar que envía al mismo fichero). Al volver a cargar se recuerdan todos los datos **excepto los descuentos**. Tipo marcado por defecto: General.
2 (3 p)	`Ejercicio2.php`: validar en el servidor. Todos los campos excepto descuentos son obligatorios (**sin `required`**). **Nº de entradas entre 1 y 4 (**sin min/max**). 'Mayor de 60' incompatible con 'Familia Numerosa'. Solo si no hay errores, mostrar el TICKET DE COMPRA con nombre, tipo, nº de entradas, descuentos separados por `/` y total a pagar.
3 (4 p)	`Ejercicio3.php`: clase `Entrada`; al comprar (datos correctos) crear un objeto `Entrada` y guardarlo en un fichero; mostrar **siempre** una tabla con todas las entradas guardadas, aunque esté vacía.

Reglas de cálculo

Concepto	Valor
Precio por entrada	General 10 euros, Mayor de 60: 8 euros, Menor de 6 años: 5 euros.
Descuentos (se multiplican)	Familia Numerosa 5% (x0.95), Abonado 10% (x0.90), Día del Espectador 2% (x0.98), Sin descuento (x1).
Ejemplo del enunciado	General, 3 entradas, Abonado + Día del Espectador: `10 * 3 * 0.90 * 0.98` = **26,46**.
Formato del fichero (ejercicio 3)	`José María Lucas;General;22/11/2024;3;Abonado/Día del Espectador;26.46` (nombre; tipo; fecha dd/mm/aaaa; nº entradas; descuentos; importe con punto). El enunciado escribe la cabecera sin el tipo, pero el ejemplo sí lo incluye: sigue el ejemplo.

Ejercicio 1 - Ejercicio1.php (recordar datos)

```
<?php
// Ejercicio1.php - Venta de entradas: formulario que recuerda los datos (salvo los descuentos)
date_default_timezone_set('Europe/Madrid');

// Datos fijos de los desplegables
$espectaculos    = ['Concierto', 'Teatro', 'Cine'];
$listaDescuentos = ['Sin descuento', 'Familia Numerosa', 'Abonado', 'Día del Espectador'];

// Escapa texto antes de mostrarlo en el HTML (evita XSS)
function e(string $t): string { return htmlspecialchars($t, ENT_QUOTES, 'UTF-8'); }

// Valores por defecto de la primera carga
$nombre = '';  $tipo = 'general';  $espectaculo = '';  $fecha = date('Y-m-d');  $entradas = '';
$descuentos = [];                                  // array con los descuentos marcados (no se recuerdan)
$mensajeError = '';

try {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {   // botón Comprar pulsado
        $nombre      = trim($_POST['nombre'] ?? '');
        $tipo        = $_POST['tipo'] ?? '';       // radio: no llega si no hay ninguno marcado
        $espectaculo = $_POST['espectaculo'] ?? '';
        $fecha       = $_POST['fecha'] ?? '';
        $entradas    = trim($_POST['entradas'] ?? '');
        $descuentos  = $_POST['descuentos'] ?? []; // select múltiple: array (o no existe)
        if (!is_array($descuentos)) { throw new UnexpectedValueException('Descuentos con formato
            incorrecto'); }
    }
} catch (Throwable $ex) {
    $mensajeError = 'No se pudieron leer los datos del formulario.';
}
?>
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Cultura Navalmoral</title></head>
<body>
<h1>CULTURA NAVALMORAL</h1>
<?php if ($mensajeError): ?><p style="color:red"><?= e($mensajeError) ?></p><?php endif; ?>
```

```
<form action="Ejercicio1.php" method="post">
  <fieldset><legend>Venta de Entradas</legend>
    <label>Nombre Completo:<br/><input type="text" name="nombre" value="<?= e($nombre) ?>"></label><br/><br/>

    Tipo Entrada:<br/>
    <label><input type="radio" name="tipo" value="general" <?= $tipo === 'general' ? 'checked' : '' ?>>
        General</label>
    <label><input type="radio" name="tipo" value="mayor60" <?= $tipo === 'mayor60' ? 'checked' : '' ?>> Mayor
        de 60</label>
    <label><input type="radio" name="tipo" value="menor6"  <?= $tipo === 'menor6'  ? 'checked' : '' ?>> Menor
        de 6 años</label>
    <br/><br/>

    Espectáculo:<br/>
    <select name="espectaculo">
      <?php foreach ($espectaculos as $esp): ?>
        <option value="<?= e($esp) ?>" <?= $espectaculo === $esp ? 'selected' : '' ?>><?= e($esp) ?></option>
      <?php endforeach; ?>
    </select><br/><br/>

    <label>Fecha del Evento:<br/><input type="date" name="fecha" value="<?= e($fecha) ?>"></label><br/><br/>
    <label>Número de Entradas:<br/><input type="number" name="entradas" value="<?= e($entradas)
        ?>"></label><br/><br/>

    Descuentos<br/>
    <select name="descuentos[]" multiple size="4">     <!-- [] => PHP lo recibe como array -->
      <?php foreach ($listaDescuentos as $d): ?>
        <option value="<?= e($d) ?>"><?= e($d) ?></option>   <!-- sin selected: no se recuerdan -->
      <?php endforeach; ?>
    </select><br/><br/>

    <input type="submit" name="comprar" value="Comprar">
  </fieldset>
</form>
</body>
</html>
```

DETALLE: La fecha por defecto solo se pone la primera vez (`date('Y-m-d')`); después se recuerda la que el usuario eligió. Si el campo número fuese `type="text"`, la validación del ejercicio 2 también debe controlar que sea un número.

Ejercicio 2 - Ejercicio2.php (validar y ticket)

Se copia el fichero anterior y se añaden, **antes del HTML**, las constantes y funciones de validación y cálculo. Parte PHP completa (el HTML del formulario no cambia salvo que se añade la zona de errores y el ticket):

```
<?php
// Ejercicio2.php - Validación en servidor y ticket de compra
date_default_timezone_set('Europe/Madrid');

const PRECIO_BASE = ['general' => 10, 'mayor60' => 8, 'menor6' => 5];            // euros por entrada
const NOMBRE_TIPO = ['general' => 'General', 'mayor60' => 'Mayor de 60', 'menor6' => 'Menor de 6 años'];
const DTO = ['Sin descuento' => 0.00, 'Familia Numerosa' => 0.05, 'Abonado' => 0.10, 'Día del Espectador' =>
    0.02];
const MIN_ENTRADAS = 1;
const MAX_ENTRADAS = 4;

$espectaculos    = ['Concierto', 'Teatro', 'Cine'];
$listaDescuentos = array_keys(DTO);

function e(string $t): string { return htmlspecialchars($t, ENT_QUOTES, 'UTF-8'); }

/** Recoge los datos del POST en un array (sin validar). */
function recogerDatos(array $post): array {
    $desc = $post['descuentos'] ?? [];
    return [
        'nombre'      => trim($post['nombre'] ?? ''),
        'tipo'        => $post['tipo'] ?? '',
        'espectaculo' => $post['espectaculo'] ?? '',
        'fecha'       => $post['fecha'] ?? '',
        'entradas'    => trim($post['entradas'] ?? ''),
        'descuentos'  => is_array($desc) ? array_values(array_intersect(array_keys(DTO), $desc)) : [],  //
            solo válidos
    ];
}

/** Valida los datos. Devuelve un array con los mensajes de error (vacío = todo correcto). */
function validar(array $d, array $espectaculos): array {
    $errores = [];
    if ($d['nombre'] === '') { $errores[] = 'El nombre completo es obligatorio.'; }

    if ($d['tipo'] === '') { $errores[] = 'Debes elegir un tipo de entrada.'; }
```

```
    elseif (!isset(PRECIO_BASE[$d['tipo']])) { $errores[] = 'El tipo de entrada no es válido.'; }

    if ($d['espectaculo'] === '') { $errores[] = 'Debes elegir un espectáculo.'; }
    elseif (!in_array($d['espectaculo'], $espectaculos, true)) { $errores[] = 'El espectáculo no es válido.';
        }

    $f = DateTime::createFromFormat('Y-m-d', $d['fecha']);
    if ($d['fecha'] === '') { $errores[] = 'La fecha del evento es obligatoria.'; }
    elseif (!$f || $f->format('Y-m-d') !== $d['fecha']) { $errores[] = 'La fecha del evento no es válida.'; }

    if ($d['entradas'] === '') { $errores[] = 'El número de entradas es obligatorio.'; }
    elseif (filter_var($d['entradas'], FILTER_VALIDATE_INT) === false) {
        $errores[] = 'El número de entradas debe ser un número entero.';
    } elseif ((int)$d['entradas'] < MIN_ENTRADAS || (int)$d['entradas'] > MAX_ENTRADAS) {
        $errores[] = 'El número de entradas debe estar entre ' . MIN_ENTRADAS . ' y ' . MAX_ENTRADAS . '.';
    }

    if ($d['tipo'] === 'mayor60' && in_array('Familia Numerosa', $d['descuentos'], true)) {
        $errores[] = 'La entrada de mayor de 60 no es compatible con el descuento de familia numerosa.';
    }
    return $errores;
}

/** Calcula el total: precio * nº entradas * (1 - descuento) de cada descuento marcado. */
function calcularTotal(string $tipo, int $n, array $descuentos): float {
    $total = PRECIO_BASE[$tipo] * $n;
    foreach ($descuentos as $dto) { $total *= 1 - (DTO[$dto] ?? 0); }
    return round($total, 2);
}

/** Texto de descuentos para mostrar/guardar: sin "Sin descuento", separados por "/". */
function textoDescuentos(array $descuentos): string {
    $reales = array_diff($descuentos, ['Sin descuento']);
    return $reales ? implode('/', $reales) : 'Ninguno';
}

// ---------------- Programa principal ----------------
$datos = ['nombre' => '', 'tipo' => 'general', 'espectaculo' => '', 'fecha' => date('Y-m-d'),
          'entradas' => '', 'descuentos' => []];
$errores = [];
$ticket = null;

try {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $datos = recogerDatos($_POST);
        $errores = validar($datos, $espectaculos);
        if (!$errores) {                                          // solo si todo es correcto
            $total = calcularTotal($datos['tipo'], (int)$datos['entradas'], $datos['descuentos']);
            $ticket = [
                'Nombre'         => $datos['nombre'],
                'Tipo de entrada'=> NOMBRE_TIPO[$datos['tipo']],
                'Nº de entradas' => (int)$datos['entradas'],
                'Descuentos'     => textoDescuentos($datos['descuentos']),
                'Total a pagar'  => number_format($total, 2, ',', '.'),
            ];
        }
    }
} catch (Throwable $ex) {
    $errores[] = 'Se ha producido un error inesperado. Inténtalo de nuevo.';
}
?>
```

En el HTML se usan `$datos['nombre']`, `$datos['tipo']`... para recordar los valores (como en el ejercicio 1) y, justo encima o debajo del formulario:

```
<?php if ($errores): ?>
  <div style="color:red"><strong>Corrige los siguientes errores:</strong>
    <ul><?php foreach ($errores as $err): ?><li><?= e($err) ?></li><?php endforeach; ?></ul>
  </div>
<?php endif; ?>

<?php if ($ticket): ?>
  <h2>TICKET DE COMPRA</h2>
  <table border="1" cellpadding="4">
    <?php foreach ($ticket as $campo => $valor): ?>
      <tr><th align="left"><?= e($campo) ?></th><td><?= e((string)$valor) ?></td></tr>
    <?php endforeach; ?>
  </table>
<?php endif; ?>
```

PRUEBAS: Comprobaciones del ejemplo: General, 3 entradas, Abonado y Día del Espectador -> 10*3 = 30; 30*0.90 = 27; 27*0.98 = 26,46. Si eliges 'Mayor de 60' + 'Familia Numerosa' debe salir el error de incompatibilidad y **no** el ticket. Prueba también: nombre vacío, entradas = 0, 5, 'abc' y vacío.

Ejercicio 3 - clase Entrada y fichero

Entrada.php (la clase, en su propio fichero):

```
<?php
// Entrada.php - Representa una entrada vendida
class Entrada {
    public function __construct(
        private string $nombre,
        private string $tipo,
        private string $fecha,          // dd/mm/aaaa
        private int $numEntradas,
        private array $descuentos,
        private float $importe
    ) {
        $this->nombre = str_replace([';', "\r", "\n"], ' ', $nombre);   // ; separa campos: no puede aparecer
            dentro
    }

    /** Línea para el fichero: nombre;tipo;fecha;nº;descuentos;importe */
    public function toLinea(): string {
        $desc = $this->descuentos ? implode('/', $this->descuentos) : 'Ninguno';
        return implode(';', [$this->nombre, $this->tipo, $this->fecha, $this->numEntradas,
                             $desc, number_format($this->importe, 2, '.', '')]);
    }

    /** Crea una Entrada a partir de una línea del fichero. Lanza excepción si el formato es incorrecto. */
    public static function desdeLinea(string $linea): Entrada {
        $c = explode(';', $linea);
        if (count($c) !== 6) { throw new UnexpectedValueException("Línea con formato incorrecto: $linea"); }
        $desc = ($c[4] === 'Ninguno') ? [] : explode('/', $c[4]);
        return new Entrada($c[0], $c[1], $c[2], (int)$c[3], $desc, (float)$c[5]);
    }

    public function getNombre(): string { return $this->nombre; }
    public function getTipo(): string { return $this->tipo; }
    public function getFecha(): string { return $this->fecha; }
    public function getNumEntradas(): int { return $this->numEntradas; }
    public function getDescuentos(): string { return $this->descuentos ? implode('/', $this->descuentos) :
        'Ninguno'; }
    public function getImporte(): float { return $this->importe; }
}
```

Ejercicio3.php: se copia el Ejercicio2.php (constantes y funciones `e`, `recogerDatos`, `validar`, `calcularTotal`, `textoDescuentos`, formulario, errores y ticket) y se añaden estas partes:

```
<?php
require_once __DIR__ . '/Entrada.php';          // require: sin la clase no se puede continuar
const FICHERO = __DIR__ . '/entradas.txt';

/** Guarda una entrada al final del fichero. */
function guardarEntrada(Entrada $entrada): void {
    $ok = file_put_contents(FICHERO, $entrada->toLinea() . PHP_EOL, FILE_APPEND | LOCK_EX);
    if ($ok === false) { throw new RuntimeException('No se pudo guardar la entrada en el fichero.'); }
}

/** Lee todas las entradas guardadas. Si el fichero aún no existe devuelve un array vacío. */
function leerEntradas(): array {
    if (!file_exists(FICHERO)) { return []; }
    $lineas = file(FICHERO, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
    if ($lineas === false) { throw new RuntimeException('No se pudo leer el fichero de entradas.'); }
    $entradas = [];
    foreach ($lineas as $l) {
        try { $entradas[] = Entrada::desdeLinea($l); }
        catch (UnexpectedValueException $ex) { /* línea corrupta: se ignora y se sigue con las demás */ }
    }
    return $entradas;
}

// ---------------- Programa principal (sustituye al del ejercicio 2) ----------------
$datos = ['nombre' => '', 'tipo' => 'general', 'espectaculo' => '', 'fecha' => date('Y-m-d'),
          'entradas' => '', 'descuentos' => []];
$errores = [];  $ticket = null;  $listado = [];

try {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
        $datos = recogerDatos($_POST);
        $errores = validar($datos, $espectaculos);
        if (!$errores) {
            $total = calcularTotal($datos['tipo'], (int)$datos['entradas'], $datos['descuentos']);
            $fechaEsp = DateTime::createFromFormat('Y-m-d', $datos['fecha'])->format('d/m/Y');   // dd/mm/aaaa
            $entrada = new Entrada($datos['nombre'], NOMBRE_TIPO[$datos['tipo']], $fechaEsp,
                                   (int)$datos['entradas'], array_diff($datos['descuentos'], ['Sin
                                       descuento']),
                                   $total);
            guardarEntrada($entrada);                          // crear objeto + guardar en el fichero
            $ticket = [ /* igual que en el ejercicio 2 */ ];
        }
    }
} catch (Throwable $ex) {
    $errores[] = 'No se pudo completar la compra: ' . $ex->getMessage();
}

try {
    $listado = leerEntradas();                                 // la tabla se muestra SIEMPRE
} catch (Throwable $ex) {
    $errores[] = 'No se pueden mostrar las entradas vendidas.';
}
?>
```

Y en el HTML, al final del `<body>`, la tabla (se pinta siempre, aunque `$listado` esté vacío):

```
<h2>Entradas vendidas</h2>
<table border="1" cellpadding="4">
  <tr><th>Cliente</th><th>Tipo</th><th>Fecha</th><th>Nº entradas</th><th>Descuentos</th><th>Importe</th></tr>
  <?php foreach ($listado as $en): ?>
    <tr>
      <td><?= e($en->getNombre()) ?></td>
      <td><?= e($en->getTipo()) ?></td>
      <td><?= e($en->getFecha()) ?></td>
      <td><?= $en->getNumEntradas() ?></td>
      <td><?= e($en->getDescuentos()) ?></td>
      <td><?= number_format($en->getImporte(), 2, ',', '.') ?> &euro;</td>
    </tr>
  <?php endforeach; ?>
  <?php if (!$listado): ?><tr><td colspan="6">Todavía no se ha vendido ninguna entrada.</td></tr><?php endif;
      ?>
</table>
```

ENTREGA: Estructura de carpeta a entregar: `2425RECUPT2/` con `Ejercicio1.php`, `Ejercicio2.php`, `Ejercicio3.php`, `Entrada.php` y el fichero de entradas que se genere. Al terminar: comprimir la carpeta y subirla a Classroom.

12. Errores típicos, erratas y lista de comprobación

12.1 'No funciona' - qué mirar primero

```
<?php
ini_set('display_errors', '1');         // en desarrollo: ver los errores en pantalla
error_reporting(E_ALL);                 // (pantalla en blanco = error fatal que no se muestra)
```

Síntoma	Causa habitual y solución
Pantalla en blanco	Error fatal (llave sin cerrar, `;` olvidado, `require` de un fichero que no existe). Activa `display_errors` o mira el log de `C:\xampp\apache\logs\error.log`.
Se ve el código PHP en el navegador	Abriste el fichero directamente. Usa `http://localhost/...`, con Apache arrancado en XAMPP, y extensión `.php`.
Undefined array key / Undefined variable	Lees `$_POST['x']` sin que exista (primera carga, radio o checkbox sin marcar). Usa `$_POST['x'] ?? ''` o `isset()`.
Warning: headers already sent	Hiciste `header()`, `session_start()` o `setcookie()` después de enviar salida (HTML, echo, un espacio antes de `<?php`). Ponlo al principio del fichero.
Cannot redeclare función/clase	Incluiste el mismo fichero dos veces. Usa `require_once`.
Los datos del formulario no llegan	Falta el atributo `name`; `method` no coincide con `$_POST`/`$_GET`; el campo está fuera del `<form>`; en subida de ficheros falta `enctype`.
Los errores de validación no se muestran	Comprueba: 1) que lees el POST **antes** de validar (`$_SERVER['REQUEST_METHOD'] === 'POST'`); 2) que el array de errores existe antes (`$errores = [];`) y la validación **añade** con `$errores[] = ...` (no lo pisa con `=`); 3) que el `foreach`/`echo` de errores se ejecuta **después** de validar y está dentro del HTML visible; 4) que el nombre del campo (`name`) es el mismo que usas en PHP; 5) si el valor es `"0"`, `empty()` falla.
El formulario se vacía al enviar	No pones `value="<?= ... ?>"`, `checked` o `selected` con los datos recibidos.
Un `if` siempre da true/false raro	Usaste `=` en vez de `==`/`===`, o `empty()` con `"0"`. Con `strpos` compara con `!== false`.
El número sale con punto en vez de coma	`number_format($n, 2, ',', '.')` para mostrar; para guardar en fichero usa punto sin separador de miles.
Tildes mal (Ã¡, ■)	Fichero PHP o de datos no guardado en UTF-8 y falta `<meta charset="UTF-8">` en el HTML.
El fichero de texto no se crea o no se actualiza	Ruta relativa mal calculada (usa `__DIR__ . '/fichero.txt'`), carpeta sin permisos, o abriste con modo `w` (borra todo) en vez de `a`.
Se duplican registros al pulsar F5	Se reenvía el POST. Redirige tras guardar (patrón Post-Redirect-Get) si el enunciado no pide recordar datos.

12.2 Erratas y detalles de las diapositivas

Diapositiva	Corrección
Variables	El nombre empieza por `$` seguido de letra o `_` (no vale un dígito).
Variables de variables	`$var1="var2":;` está mal escrito: es `$var1 = "var2";`.
Funciones de cadenas	Se escriben `strncmp`, `strncasecmp`, `strnatcasecmp`, `rtrim` (no 'Retrim'), `chop` (alias de rtrim). `split()` **ya no existe** en PHP 7/8: usa `explode()` o `preg_split()`. `strtolower` etc. van en minúsculas.
Funciones de ficheros	`is_writable` (no `is_writedable`), y sirve para saber si se puede **escribir** (la diapositiva repite la definición de legible).
Excepciones	La clase es `Throwable` (no 'Trowable').
Procesamiento del formulario	Chequear el botón: `isset($_POST['Enviar'])` (corchetes, no paréntesis).
Sentencia `fopen` modo `a+`	En `a` y `a+` el puntero está al final y **siempre escribe al final** aunque hagas `fseek`.
Boolean en la tabla de tipos	Son false: `0`, `0.0`, `""`, `"0"`, `[]`, `null`. Cualquier otro valor es true.
Ejercicio 7	Dice 'entre 1 y 10' pero `rand(0,10)`: usa `rand(1, 10)`.
Ejercicio 11	'Serie x2' es ambiguo: potencias de 2 (2, 4, 8, ...) o cuadrados (1, 4, 9, ...). Pregunta o indica cuál has elegido.

12.3 Lista de comprobación antes de entregar el examen

- Carpeta con el nombre exacto que piden y **todos** los ficheros dentro (incluida la clase y el fichero de datos). Comprimir y subir a Classroom.
- Cada ejercicio funciona **abriéndolo desde `http://localhost/...`**, no desde el explorador de archivos.
- Probar el camino feliz y los errores: campos vacíos, 0, negativos, texto en campos numéricos, ningún radio marcado, ningún descuento, fichero inexistente.
- No usar lo que el enunciado prohíbe (`required`, `min`, `max`...). Validar **en PHP**.
- `try/catch` alrededor de lo que puede fallar (ficheros, creación de objetos, lectura del POST). Mensajes claros al usuario, sin rutas ni trazas.
- Todo texto del usuario mostrado con `htmlspecialchars` (también en `value=""`).
- Código modular: funciones pequeñas con nombre claro (`validar`, `calcularTotal`, `guardarEntrada`...), constantes en vez de números 'mágicos' (precios, límites).
- Comentarios útiles: una línea por función y por bloque importante.
- Si el enunciado pide una clase, **úsala de verdad** (crear objeto con `new`, métodos, ficheros separados) y no la dejes sin usar.
- Fichero de datos con el formato exacto que pide el enunciado (separador `;`, fecha dd/mm/aaaa, `/` entre descuentos, etc.).
- La tabla de resultados se muestra aunque esté vacía si así lo pide.

12.4 Esqueleto reutilizable para cualquier ejercicio de formulario

```
<?php
// 1. Constantes y datos fijos            const PRECIO = ...;   $opciones = [...];
// 2. Funciones (modularidad)             e(), recogerDatos(), validar(), calcular(), guardar(), leer()
// 3. Valores por defecto                 $datos = [... 'fecha' => date('Y-m-d')];  $errores = [];
// 4. Programa principal
try {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $datos   = recogerDatos($_POST);           // leer con ??
        $errores = validar($datos);                // validar en servidor
        if (!$errores) {                           // solo si no hay errores:
            // calcular, crear objeto, guardar en fichero, preparar el resultado a mostrar
        }
    }
    // leer ficheros para mostrar tablas
} catch (Throwable $ex) {
    $errores[] = 'Error inesperado: ' . $ex->getMessage();
}
?>
<!-- 5. HTML: errores, resultado, formulario con valores recordados, tabla -->
```